

面向资源约束量子布尔函数综合的神经蒙特卡洛树搜索方法

中文论文稿 v11: 主线收束版

2026 年 7 月 8 日

摘要

量子布尔函数 oracle 综合是 Grover 搜索、密码分析和若干量子算法子程序中的基础编译步骤。容错量子计算中 T 门和多控门分解代价较高, 因而 oracle 的 T-count、CNOT、深度和辅助比特会直接影响逻辑层资源估计。本文将布尔 oracle 综合表述为代数正规形 (algebraic normal form, ANF) 和固定极性 Reed-Muller (fixed-polarity Reed-Muller, FPRM) 项集合上的资源加权搜索问题, 提出结合神经动作先验、蒙特卡洛树搜索、布尔环线性因子筛选、结构级 depth policy 与保守 guard 的 Resource-NMCTS 框架。该方法不依赖 SSHR 的 parallelotope cover 空间; SSHR 仅作为 CNOT-oriented small-scale baseline 使用。实验表明, 在 $n \leq 6$ 的 177 个传统函数上, Pareto-Resource-NMCTS 相对 ESOP cube beam 获得 174/0/3 的 score 胜/负/平, 平均 score 降低 36.09%; 相对 weighted ESOP-MILP 获得 167/3/7, 平均 score 降低 29.84%。在 $n = 18$ 的 12 个随机 ANF 函数上, adaptive depth-2 Boolean-ring screen 相对 single screen 平均 score 降低 6.47%, 完整 Resource-NMCTS 相对 adaptive screen 进一步降低 23.85%。在 $n = 20$ 边界函数上, screen-gated Resource-NMCTS 与完整 Resource-NMCTS 资源完全持平, 并把平均运行时间从 77.10 s 降至 20.71 s; 在独立 $n = 19, 20$ gate holdout 中, screen-gated Resource-NMCTS 在 16 个函数上全部 score 持平并平均节省 36.83% 时间。在 $n = 20, 22, 24, 28$ 的 192 个项集级 scale 测试中, depth policy 相对 single screen 获得 169/0/23, 平均 score 降低 6.56%; 相对 all-depth adaptive 只损失 0.08% 平均 score, 并节省 31.01% 平均时间。进一步在 $n = 32, 36, 40$ 的 144 个项集上, depth policy 相对 single screen 获得 110/0/34, 平均 score 降低 5.55%, 并与 all-depth adaptive 全部 score 持平、平均节省 33.14% 时间。高预算 depth-frontier 实验显示, depth-4 Boolean-ring screen 相对 fixed depth-2 在 $n = 20, 28, 40$ 上获得 49/0/23, 平均 score 进一步降低 3.10%, 但平均运行时间增加 682.97%。上述项集级实验中, 3000/3000 个方法行均通过 factor plan 的 ANF 展开验证和 emitted X/CNOT/MCT circuit 的 GF(2) 多项式符号模拟验证。结果支持一条独立于 SSHR 的资源约束 oracle 综合主线, 同时也明确边界: 本文只讨论逻辑层综合, 不包含硬件 mapping; $n > 20$ 结果是项集级符号验证, 不是完整 truth-table simulation; 当前高维 learned prior 的独立质量收益仍弱, 结构级 AI 仍需从安全门控推进到更大幅度的质量收益。

关键词: 量子 oracle 综合; 布尔函数综合; 神经蒙特卡洛树搜索; ANF; FPRM; 布尔环; 资源约束综合

1 引言

给定布尔函数 $f: \{0, 1\}^n \rightarrow \{0, 1\}$, 量子 oracle 通常实现为

$$|x\rangle|y\rangle \mapsto |x\rangle|y \oplus f(x)\rangle. \quad (1)$$

这一变换是许多量子算法调用经典谓词的入口。若 f 被直接展开为单项式并逐项写入输出位，线路实现简单且容易验证，但会忽略项之间的可共享结构；若过度追求共享，又可能增加辅助比特、CNOT 和调度深度。因此，量子布尔函数综合不适合只写成“最小门数”问题，而应作为多资源约束下的组合搜索问题处理。

本文的出发点是 ANF 表示。若

$$f(x) = \bigoplus_{m \in \mathcal{M}} \prod_{i \in m} x_i, \quad (2)$$

则每个单项式都可转化为受控乘法结构，并异或到输出位。ANF 的优势是语义透明；弱点是它不自动暴露公共因子、线性奇偶量、极性重写和递归子问题。已有 XAG、ROS、BDD 和 SSHR 等路线分别从 XOR/AND 图、资源约束 LUT 映射、决策图和 parallelotope cover 改进 oracle 或可逆逻辑综合 [2, 3, 4, 6]。这些工作说明中间表示会显著改变可优化资源，但仍留下一个问题：能否直接在 ANF/FPRM 项集合上搜索可复用代数结构，并用 AI 策略在该搜索空间中选择更优动作？

本文的一句话论证是：在逻辑层量子布尔函数 oracle 综合中，ANF/FPRM 项集合上的资源感知神经搜索，配合布尔环线性因子筛选和保守结构门控，可以在传统函数与高维随机项集合上降低 T-count 和加权逻辑资源；当前边界是该方法尚未覆盖硬件 mapping、真实噪声模型和完整的大规模 truth-table 语义验证。本文不把目标写成“AI-SSHR”，因为方法本身并不继承 SSHR 的 parallelotope 空间。SSHR 在本文中只是小规模 CNOT 导向基线，用于暴露 T-count 与 CNOT 指标之间的取舍。

2 相关工作与本文定位

2.1 布尔表示与 oracle 综合

Xor-And-Inverter Graphs(XAG)把 XOR 与 AND 结构分离,使 AND 节点数量与非 Clifford 成本直接相关。Meuli 等人将 XAG 用于 quantum compilation[2], 并从 multiplicative complexity 角度解释低 T-count oracle 的结构来源 [1]。ROS 进一步把 oracle synthesis 建模为 resource-constrained LUT mapping[3]。这些工作与本文的共同点是把布尔结构视为资源优化入口；不同点是，本文不固定使用 XAG 或 LUT，而是在 ANF/FPRM 项集合上搜索因子分解与 compute/uncompute 结构。

BDD-based reversible synthesis 用决策图处理较大布尔函数 [4]。Back-end-aware oracle synthesis 则把 XAG 优化与后端质量指标连接起来 [5]。本文与这些工作互补：当前实验只报告逻辑层 T-count、CNOT、depth、gate count 和辅助比特，不报告设备连通性、routing、magic-state factory 或噪声下保真度。

SSHR 使用 parallelotope 的空间结构做 small-scale Boolean function 的 CNOT-oriented synthesis[6]。本文复现并保留 SSHR-H/SSHR-I 作为小规模比较对象，但方法本体不枚举 parallelotope candidate，也不以 CNOT-only 最优为唯一目标。更准确的定位是：本文是一条面向低 T-count 和加权逻辑资源的 ANF/FPRM 搜索路线，SSHR 是其中一个能够说明 CNOT trade-off 的外部参照。

表 1: 本文术语与边界。

术语	本文含义
Resource-NMCTS	在 ANF/FPRM 项集合上进行资源加权搜索的主框架, 包括神经先验、MCTS、guard 和 portfolio 选择。
Boolean-ring screen	在 square-free Boolean ring 中搜索可复用线性因子的结构筛选分支, 允许线性因子与 quotient 共享变量。
Depth policy	预测 single、depth-1 或 depth-2 screen 的结构级神经策略。
Skip guard	判断是否安全跳过更深 screen 或 Resource tail 的保守门控; 当前目标优先是 zero-loss。
SSHR	CNOT-oriented small-scale baseline; 本文方法不从 SSHR 搜索空间出发。

2.2 学习式搜索与量子线路优化

学习式搜索已经用于若干离散综合问题。ShortCircuit 使用 AlphaZero 风格搜索设计经典布尔电路 [7]; Gumbel AlphaZero 被用于 Clifford+T unitary synthesis[8]; ZX-calculus 优化中出现了强化学习和图神经网络驱动的 rewrite-rule 选择 [9]; MonteQ 使用 MCTS 处理量子线路综合中的动作排序 [10]。这些工作说明学习式策略适合组合空间, 但它们的状态和动作并不是 Boolean oracle 的 ANF/FPRM 因子分解。本文的 AI 贡献应被理解为面向 Boolean oracle 结构的动作排序、结构深度选择和安全门控, 而不是泛化的“RL 直接生成量子线路”。

3 问题定义与资源模型

本文输入为布尔函数的 truth-table 或 ANF 表示, 输出为实现式 (1) 的逻辑层可逆线路。候选线路由 X、CNOT 和多控 Toffoli (MCT) 抽象门组成, 并在资源统计阶段转化为 T-count、CNOT、深度、总 gate count 和 peak ancilla。用于搜索排序的统一分数为

$$\text{score} = T + 0.04 \text{ CNOT} + 0.015 \text{ depth} + 0.01 \text{ gates} + 2 \text{ ancilla}. \quad (3)$$

该分数只用于逻辑层候选选择, 不代表某个具体硬件平台的物理开销。为避免单一分数掩盖取舍, 实验表同时报告各项资源。

在 $n \leq 20$ 的函数级实验中, 本文对候选线路做完整 truth-table 语义验证。在 $n > 20$ 的项集级 scale 实验中, 完整 truth-table 构造本身不可接受, 因此采用两层符号验证: 第一层把 factor plan 展开回 ANF 项集合, 检查目标项集合等价; 第二层对 emitted X/CNOT/MCT circuit 做 GF(2) 多项式符号模拟, 检查输入线复原、输出线等价和辅助线清零。该验证不等同于逐点枚举 2^n 个输入, 但比只统计 plan 资源更强, 因为它验证了生成线路的代数语义。

4 方法

4.1 状态、动作与资源感知搜索

Resource-NMCTS 的状态是当前尚未综合的项集合 $\mathcal{M}$ 。一次动作选择一个可计算、可反算的局部代数结构, 将 $\mathcal{M}$ 分解为 quotient 子问题和 rest 子问题, 再递归生成 compute/action/uncompute

形式的线路。候选动作包括公共单项式因子、FPRM 极性重写后的公共项、线性奇偶因子、布尔环线性因子，以及由神经模型扩展的 root actions。

神经动作先验并不直接输出量子门序列，而是对候选因子动作排序。特征包括候选覆盖率、项数变化、项平均次数、变量覆盖密度、即时资源收益和子问题规模等结构统计。MCTS 在有限预算内探索候选组合，rollout 使用确定性资源估计与已有 baseline 候选。最终选择由 baseline-preserving guard 兜底：若学习候选不优于确定性 baseline，则返回 baseline。这个设计牺牲部分探索自由度，但能显著降低 AI 负迁移造成资源退化的风险。

4.2 布尔环线性因子

早期 linear-pair 分支主要搜索形如

$$(x_i \oplus x_j \oplus \dots) \cdot h(x) \quad (4)$$

的结构，并要求线性因子变量与 quotient $h(x)$ 的变量不相交。该约束便于实现，却排除了 square-free Boolean ring 中合法的重叠变量情形。本文的 Boolean-ring screen 允许二者共享变量，并在展开时使用 $x_i^2 = x_i$ 与 GF(2) 偶数项消去。例如

$$(x_i \oplus x_j)x_im = x_im \oplus x_ix_jm. \quad (5)$$

线路层面仍可先用 CNOT 计算线性奇偶量，再用该辅助量控制 quotient 子线路。因此，布尔环线性因子不是简单扩大 beam 宽度，而是把原先被实现约束排除的代数候选重新纳入可综合搜索空间。

4.3 结构级 AI 策略

高维实验显示，fixed single screen、depth-1 screen 和 depth-2 screen 的收益依赖项集合结构。本文因此训练 depth policy，输入项数、次数分布、变量覆盖密度、二元共现密度和 root Boolean-ring action 统计，输出应该使用的 screen 深度。监督标签来自实际运行三种 screen 后按式 (3) 选择的最佳 depth。

由于固定 depth-2 screen 是很强的质量 baseline，本文又训练保守 depth-2 skip guard。Static-direct 模式只使用项集合静态特征，若预测 depth-1 足以与 fixed depth-2 持平，则直接运行较浅 screen；否则运行 depth-2。Shallow-staged 模式先运行 single/depth-1 screen，再用浅层观测判断是否跳过 depth-2。阈值按 zero-false-skip 约束选择，使门控优先避免质量损失。

此外， $n = 20$ 边界切片中完整 Resource-NMCTS 的 Resource tail 与 adaptive screen 资源持平但运行更慢。为控制该长尾，本文训练 screen-gate 判断是否可以跳过 Resource tail。该门控当前只作为运行时控制证据：它说明某些切片上能以零资源损失节省时间，但还不能说明高维 Resource tail 普遍不必要。

5 实验设置

实验分为五组。第一组使用 $n \leq 6$ 的 177 个传统函数，与 direct ANF、AND-direct ANF、ESOP cube beam、weighted ESOP-MILP、SSHR-H、ABC 和 BDD baseline 比较。第二组使用 $n = 18$

表 2: $n \leq 6$ 传统函数切片上的平均逻辑资源。

方法	平均 T	平均 CNOT	平均 depth	平均 ancilla	平均 score
Direct ANF	184.1	134.2	134.6	1.33	194.3
AND-direct ANF	101.0	166.5	166.9	2.18	115.2
Resource-NMCTS	43.9	89.9	94.5	1.92	53.2
Pareto-Resource-NMCTS	40.4	83.0	88.0	2.03	49.6
ESOP-MILP	83.6	133.5	159.6	2.32	96.7
SSHR-H	81.0	67.6	88.7	1.36	88.2

随机 ANF 函数, 观察 adaptive screen 与完整 Resource-NMCTS 的差距。第三组使用 $n = 20$ 边界函数, 检验在 FPRM root beam 和 fast linear-pair 超时区域中, depth-2 screen 是否仍能给出可运行改进, 并用 $n = 19, 20$ held-out 切片验证 screen-gate 是否会错误跳过 Resource tail。第四组使用 $n = 14, 16, 18$ 生成式项集训练 depth policy 与 skip guard, 并在 held-out $n = 20$ 项集上测试结构选择能力。第五组在 ANF 项集合上评估 $n = 20, 22, 24, 28$ 和扩展 $n = 32, 36, 40$ 的 screen-scale 行为, 并对每个 plan 和 emitted circuit 做符号等价验证。

所有 paired comparison 只纳入函数名或项集名匹配、语义正确且未 timeout 的结果。Timeout 行保留为运行边界证据, 但不参与正确结果均值比较。本文所有实验均为逻辑层资源估计, 不包含 hardware mapping 和 noise-aware scheduling。

6 结果

6.1 小规模传统函数

表 2 给出 $n \leq 6$ 传统函数上的平均逻辑资源。相对 direct ANF, Pareto-Resource-NMCTS 平均 T-count 降低 72.25%, 平均 score 降低 67.80%。相对 ESOP cube beam, Pareto-Resource-NMCTS 获得 174/0/3 的 score 胜/负/平, 平均 score 降低 36.09%。相对 weighted ESOP-MILP, 获得 167/3/7, 平均 score 降低 29.84%。

该结果支持本文的低 T-count 与加权资源优势, 但也给出重要边界: SSHR-H 的平均 CNOT 最低, 因此本文不能主张 CNOT-only 指标全面支配 SSHR。更准确的表述是, 本文方法在当前资源权重下用一定 CNOT 与辅助比特 trade-off 换取更低 T-count 和更低 score。

6.2 高维函数级边界

表 3 汇总 $n = 18$ 与 $n = 20$ 函数级边界结果。在 $n = 18$ 随机 ANF 函数上, adaptive depth-2 Boolean-ring screen 相对 single screen 获得 11/0/1 的 score 胜/负/平, 平均 score 从 11349.06 降至 10670.94; 完整 Resource-NMCTS 平均 score 为 7315.62, 相对 adaptive screen 进一步降低 23.85%。这说明 $n = 18$ 上快速 screen 是有效候选生成器, 但不能替代完整资源搜索。

在 $n = 20$ 边界函数上, FPRM root beam 和 fast linear-pair 均触发 300 s task timeout。Adaptive depth-2 screen 相对 single screen 获得 5/0/1, 平均 score 降低 7.47%。集成该分支后, Resource-NMCTS 相对 AND-direct ANF 获得 5/0/1, 平均 score 降低 11.80%。由于此切片中完

表 3: 高维函数级边界结果。

切片	方法	平均 T	平均 CNOT	平均 score	平均时间 (s)
$n = 18$	Single screen	10389.33	16268.42	11349.06	0.82
$n = 18$	Adaptive depth-2 screen	9767.00	15301.58	10670.94	4.39
$n = 18$	Resource-NMCTS	6639.33	11380.92	7315.62	226.18
$n = 20$	AND-direct ANF	21516.67	33635.67	23491.15	10.41
$n = 20$	Single screen	20786.00	32492.50	22695.15	10.70
$n = 20$	Adaptive depth-2 screen	19535.33	30542.50	21331.24	20.67
$n = 20$	Resource-NMCTS	19535.33	30542.50	21331.24	77.10
$n = 20$	Screen-gated Resource-NMCTS	19535.33	30542.50	21331.24	20.71

表 4: 结构级 depth policy 在 held-out $n = 20$ 项集上的比较。

比较	样本数	score 胜/负/平	平均 Δ score	平均 Δ time
Policy vs single screen	48	42/0/6	-6.57%	+2224.00%
Policy vs depth-1 screen	48	34/0/14	-2.57%	+257.27%
Policy vs depth-2 screen	48	0/5/43	+0.28%	-10.85%
Policy vs all-depth adaptive	48	0/5/43	+0.28%	-34.71%

整 Resource tail 与 adaptive screen 资源持平, screen-gated Resource-NMCTS 可以把平均时间从 77.10 s 降到 20.71 s。

6.3 结构级 AI 与门控

Depth policy 在 $n = 14, 16, 18$ 项集上训练, 并在 held-out $n = 20$ 项集上评估。表 4 显示, policy 相对 single screen 获得 42/0/6, 平均 score 降低 6.57%; 相对 depth-1 screen 获得 34/0/14, 平均 score 降低 2.57%。相对 all-depth adaptive, policy 平均 score 只高 0.28%, 但平均运行时间降低 34.71%。这说明结构级 AI 已能学习 screen 深度选择, 但尚未在 score 上超过 fixed depth-2。

为去掉这一质量缺口, 本文训练了保守 depth-2 skip guard。Static-direct 模式在 held-out $n = 20$ test 中没有 false skip, 96 个样本全部与 fixed depth-2 screen score 持平, 并相对 fixed depth-2 与 all-depth adaptive 分别节省 0.54% 和 24.74% 平均时间。Screen-gate 的独立 held-out 验证也支持保守门控: 在 $n = 19, 20$ 共 16 个函数上, screen-gated Resource-NMCTS 与完整 Resource-NMCTS 全部 score 持平, 平均节省 36.83% 时间; 若直接用 adaptive screen 替代 Resource tail, $n = 19$ 子集会出现 4/8 个 score loss。因此, 门控贡献不是简单跳过搜索, 而是在保守阈值下识别何时可以安全跳过。

6.4 项集级大规模验证

表 5 给出 $n = 20, 22, 24, 28$ 的项集级 screen-scale 结果。All-depth adaptive screen 相对 single screen 获得 169/0/23, 平均 score 降低 6.63%; 但它与 fixed depth-2 在 score 上全部持平, 并增加 47.92% 平均时间。Depth policy 相对 single screen 获得 169/0/23, 平均 score 降低 6.56%; 相对 all-depth adaptive 只有 0/6/186, 平均 score 增加 0.08%, 但平均时间降低 31.01%。在最大维度 $n = 28$ 上, depth policy 与 all-depth adaptive 在 48 个项集上全部 score 持平, 省时 32.15%。

表 5: $n = 20, 22, 24, 28$ 项集级 screen-scale 比较。所有方法行均通过 plan 和 emitted-circuit ANF 符号验证。

n	方法	baseline	score 胜/负/平	平均 Δ score	平均 Δ time
all	adaptive all-depth	single screen	169/0/23	-6.63%	+3077.71%
all	adaptive all-depth	fixed depth-2	0/0/192	+0.00%	+47.92%
all	depth policy	single screen	169/0/23	-6.56%	+2328.66%
all	depth policy	all-depth adaptive	0/6/186	+0.08%	-31.01%
28	depth policy	all-depth adaptive	0/0/48	+0.00%	-32.15%
all	direct depth-2 guard	fixed depth-2	0/0/192	+0.00%	-0.14%

为检验该结构策略是否只在 $n = 20, 22, 24, 28$ 成立, 本文进一步使用同一协议扩展到 $n = 32, 36, 40$ 。表 6 显示, depth policy 相对 single screen 获得 110/0/34, 平均 score 降低 5.55%; 相对 all-depth adaptive 在 144 个项集上全部 score 持平, 并节省 33.14% 平均时间。该结果比 $n = 20, 22, 24, 28$ 更强地说明, 训练于 $n = 14, 16, 18$ 的结构级策略可以外推到更高维项集合。

表 6: $n = 32, 36, 40$ extended screen-scale 比较。所有方法行均通过 plan 和 emitted-circuit ANF 符号验证。

n	方法	baseline	score 胜/负/平	平均 Δ score	平均 Δ time
all	adaptive all-depth	single screen	110/0/34	-5.55%	+2680.60%
all	adaptive all-depth	fixed depth-2	0/0/144	+0.00%	+64.37%
all	depth policy	single screen	110/0/34	-5.55%	+2061.11%
all	depth policy	all-depth adaptive	0/0/144	+0.00%	-33.14%
all	direct depth-2 guard	fixed depth-2	0/0/144	+0.00%	-0.00%

前述策略偏向快速或中等预算。为判断 fixed depth-2 是否已经达到质量上限, 本文又在 $n = 20, 28, 40$ 的 72 个项集上运行 depth-3/4 Boolean-ring screen。表 7 显示, depth-3 相对 fixed depth-2 获得 49/0/23, 平均 score 降低 1.93%; depth-4 相对 fixed depth-2 同样获得 49/0/23, 平均 score 降低 3.10%。这说明更深布尔环筛选确实能形成新的质量前沿, 但代价是运行时间分别增加 193.71% 和 682.97%。因此, depth-3/4 应被写成高预算质量模式, 而不是默认快速策略。

表 7: $n = 20, 28, 40$ depth-frontier screen-scale 比较。所有方法行均通过 plan 和 emitted-circuit ANF 符号验证。

n	方法	baseline	score 胜/负/平	平均 Δ score	平均 Δ time
all	screen depth-3	fixed depth-2	49/0/23	-1.93%	+193.71%
all	screen depth-4	fixed depth-2	49/0/23	-3.10%	+682.97%
all	screen depth-4	screen depth-3	44/0/28	-1.21%	+127.55%
all	all-depth adaptive depth ≤ 4	fixed depth-2	49/0/23	-3.10%	+1129.85%

主 scale、extended scale 和 depth-frontier 三组项集级结果合计包含 3000 个方法行。所有方法行均通过两层验证: factor plan 的 ANF 展开验证为 3000/3000, emitted X/CNOT/MCT circuit 的 GF(2) 多项式符号模拟验证为 3000/3000, mismatch 总数为 0。这使项集级结果不只是资源估计, 但仍应明确它不是完整 truth-table simulation。

7 讨论

本文最稳妥的贡献表述有四点。第一，方法主线独立于 SSHR。本文不是把 SSHR 改成 AI 版本，而是在 ANF/FPRM 项集合上做资源加权搜索；SSHR 的作用是 CNOT-oriented baseline。第二，布尔环线性因子提供了明确的结构扩展，允许线性因子与 quotient 共享变量，并在 $n = 18, 20$ 上稳定改善 single screen。第三，结构级 AI 已有正向证据：depth policy 学会了 screen 深度选择，保守 guard 消除了相对 fixed depth-2 的 score 缺口，并减少 all-depth adaptive overhead。第四，项集级 scale 测试将证据推进到 $n = 20, 22, 24, 28, 32, 36, 40$ ，depth-frontier 进一步显示 depth-3/4 可以打破 fixed depth-2 的质量上限，并给出 emitted-circuit 级的符号等价验证。

当前结果还不能支撑更强的结论。其一， $n = 18$ 和 held-out $n = 19$ 上完整 Resource-NMCTS 仍能超过 adaptive screen，说明中等高维仍需要更深搜索；其二， $n = 20$ 切片中完整 Resource-NMCTS 与 adaptive screen 资源持平，说明该切片的收益主要来自结构筛选而非 MCTS tail；其三，高维 learned prior 的独立质量收益仍弱，当前 AI 主体更多体现为结构选择和安全门控；其四， $n = 22$ 到 $n = 40$ 的项集级结果虽有 emitted-circuit ANF 符号等价验证，但不包含完整 truth-table 枚举。

下一步应把论文推进到三个可量化目标。第一，把 shallow-staged guard 的安全跳过能力蒸馏回 direct guard，使 held-out $n = 20$ 上继续保持不劣于 fixed depth-2 的 score，同时把 direct 模式相对 fixed depth-2 的时间优势提高到至少 5%。第二，把 screen-gate 从单一边界规则扩展为多特征分支选择器，联合预测 screen 深度、是否进入 FPRM polarity search、是否运行完整 Resource-NMCTS tail。第三，补充更多外部 reversible-toolchain 或 ROS/mockturtle 风格比较，使论文从内部资源表推进到更有说服力的工具链对比。

8 结论

本文提出了面向资源约束量子布尔函数综合的 Resource-NMCTS 方法。它以 ANF/FPRM 项集合为状态，以逻辑资源 score 为目标，用布尔环线性因子、神经动作先验、MCTS、baseline guard、depth policy 和 screen-gate 共同生成候选线路。实验表明，该方法在 $n \leq 6$ 传统函数上显著降低 T-count 与加权 score，在 $n = 18, 20$ 高维函数级切片上验证了 adaptive screen 与 Resource tail 的互补作用，在 $n = 20$ 到 $n = 40$ 项集级 scale 测试中展示了结构级 AI 的可扩展性，并通过 depth-frontier 实验证明更深 Boolean-ring screen 可在高预算下继续降低 score。当前稿件已经形成独立于 SSHR 的论文主线，但最终投稿还需要继续强化高维神经策略的质量收益、完整 truth-table 可验证切片和外部工具链比较。

参考文献

- [1] G. Meuli, M. Soeken, E. Campbell, M. Roetteler, and G. De Micheli. The role of multiplicative complexity in compiling low T-count oracle circuits. In *Proceedings of ICCAD*, 2019.
- [2] G. Meuli, M. Soeken, and G. De Micheli. Xor-And-Inverter Graphs for quantum compilation. *npj Quantum Information*, 8:7, 2022.

- [3] G. Meuli, M. Soeken, M. Roetteler, and G. De Micheli. ROS: Resource-constrained oracle synthesis for quantum computers. *Electronic Proceedings in Theoretical Computer Science*, 318:119–130, 2020.
- [4] R. Wille and R. Drechsler. BDD-based synthesis of reversible logic for large functions. In *Proceedings of DAC*, pp. 270–275, 2009.
- [5] M. Yu, A. Tempia Calvino, M. Soeken, and G. De Micheli. Back-end-aware fault-tolerant quantum oracle synthesis. In *Proceedings of ASP-DAC*, pp. 930–937, 2025.
- [6] Q. Zheng, Y. Xu, J. Zhang, Z. Su, and S. Zheng. CNOT oriented synthesis for small-scale Boolean functions using spatial structures of parallelotopes. In *Proceedings of ICCAD*, 2025.
- [7] D. Tsaras, A. Grosnit, L. Chen, Z. Xie, H. Bou-Ammar, and M. Yuan. ShortCircuit: AlphaZero-driven circuit design. *arXiv:2408.09858*, 2024.
- [8] S. Rietsch, A. Y. Dubey, C. Ufrecht, M. Periyasamy, A. Plinge, C. Mutschler, and D. D. Scherer. Unitary synthesis of Clifford+T circuits with reinforcement learning. In *IEEE International Conference on Quantum Computing and Engineering*, pp. 824–835, 2024.
- [9] J. Riu, J. Nogu  , G. Vilaplana, A. Garcia-Saez, and M. P. Estarellas. Reinforcement learning based quantum circuit optimization via ZX-calculus. *Quantum*, 9:1758, 2025.
- [10] M. Machiya, M. Menickelly, P. Hovland, and J. Liu. MonteQ: A Monte Carlo tree search based quantum circuit synthesis framework. *arXiv:2604.19029*, 2026.